1. Aim

To calculate the SHA-256 hash value of a digital evidence file and demonstrate how even a small modification changes the hash completely, thereby verifying file integrity in digital forensics.

2. Algorithm

```
1. Start the program.
2. Import the hashlib module.
3. Open the evidence file in binary read mode.
4. Create a SHA-256 hash object.
5. Read the file in blocks of 1024 bytes.
6. Update the hash object with each block.
7. Generate the final SHA-256 hash value.
8. Display the hash value.
9. Compare the hash before and after file modification.
10. Stop the program.
```

3. Code

```
import hashlib

filename = "evidence.txt"

sha = hashlib.sha256()

with open("C:/Users/saich/OneDrive/Desktop/exp2/digital forensics for.txt", "rb") as file:
    while True:
        data = file.read(1024)

        if not data:
            break

        sha.update(data)

print("SHA-256 Hash:")
print(sha.hexdigest())
```

4. Output

```
SHA-256 Hash original:
64cac5876292ab19776865bfbe117ff85dfaca05ce97645a523cc776d45ae06b

SHA-256 Hash modified:
b6104237b65b99ddea713f6e60c4c07eec968a1e69415038832f0fd6c9a9a738
```

5. Result

The SHA-256 hash values of the original and modified files are different. This proves that even a minor change in the file alters its hash value, demonstrating the effectiveness of SHA-256 for integrity verification in digital forensics.